

Experimental Analysis of In-Memory Hard Cohesion Caching for Write-Intensive Web Applications

Simon Gilbu

Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
simon.gilbu@uit.no

Thomas Tekie

Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
thomas.tekie@uit.no

***Index Terms*—caching, cache invalidation, write-intensive workloads, web applications.**

I. INTRODUCTION

Websites and applications often use caches to store copies of data, avoiding the need to re-fetch data every time it is requested, since this can be expensive and slow. A problem often occurs when the underlying data is updated, and this raises the central challenge of keeping the cache consistent with the underlying data source, without returning outdated data or adding unnecessary latency.

Much research exists on caching and caching techniques in web applications. However, we find a gap in research covering experimental analysis of hard-cohesion caching in write-heavy web applications. We hope that a highly reproducible baseline for the measured impact of hard-cohesion caching techniques can help support and strengthen existing research. More specifically, we want to look deeper at push-based invalidation for in-application caching on small, isolated systems.

What are the measured impacts of push-based cache invalidation in write-heavy web-applications?

II. RELATED WORK

Several papers discuss methods for caching and caching architectures in large networks; surveys such as [1] have been cited for decades and outline foundational approaches to web-caching architectures. [2] gives deeper insight into existing caching solutions and architectures used in large, write-intensive social media services, with CaaS sampled on a per-service basis to provide insight into real system behavior. [3] provides a methodological approach for measuring the impact of soft-cohesion caching in smaller applications. Within this existing landscape, we aim to answer a question about hard-cohesion caching in applications under heavy write-intensive workloads.

There is a lot of research on caching in large distributed systems [1], [2], but there is a gap around the experimental,

reproducible measurement of hard cohesion caching in write heavy scenarios on small systems. The closest existing work, [3], targets TTL based caching for applications that do not require instant updates, such as content that can tolerate a short delay before incoming change.

Our work focuses on applications where freshness is not optional: comparable settings would include financial systems, where a delayed update on price or balance is an error, or high frequency social platforms such as Twitter, where a delay in write is directly noticeable to the user. This is why our main focus is on freshness and hit ratio once writes become frequent, since the TTL based approach evaluated by [3] is not enough for systems that require live updates.

III. MODEL

In order to model our system, we look at creating a small read-write API over a map of some key-value paired data. With some synthetic delay in access to arbitrary storage. This is deemed sufficient to simulate the read-write patterns of database access in simple applications[3].

The delay is overcome by providing a cache within the application. Our API supports read and write operations, while our cache supports GET, SET, and EVICT operations. The caching is then baked into the API, and the API writes against an arbitrary data source on the backend (file on disk, network, etc).

A. System Architecture

The two patterns of cache behavior, in addition to the *no-cache* baseline, can be expressed by outlining the expected pattern of behavior for a sequence of operations '[SET, GET, GET, SET, GET]' over some given key 'K1' and its corresponding values 'V1, V2'.

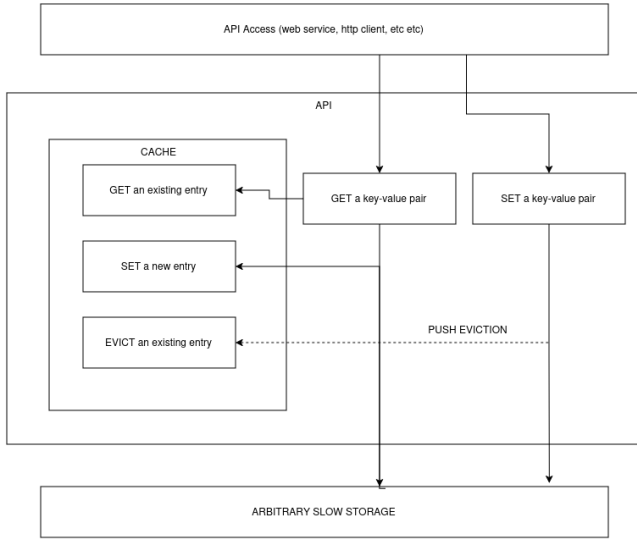


Fig. 1. Architecture

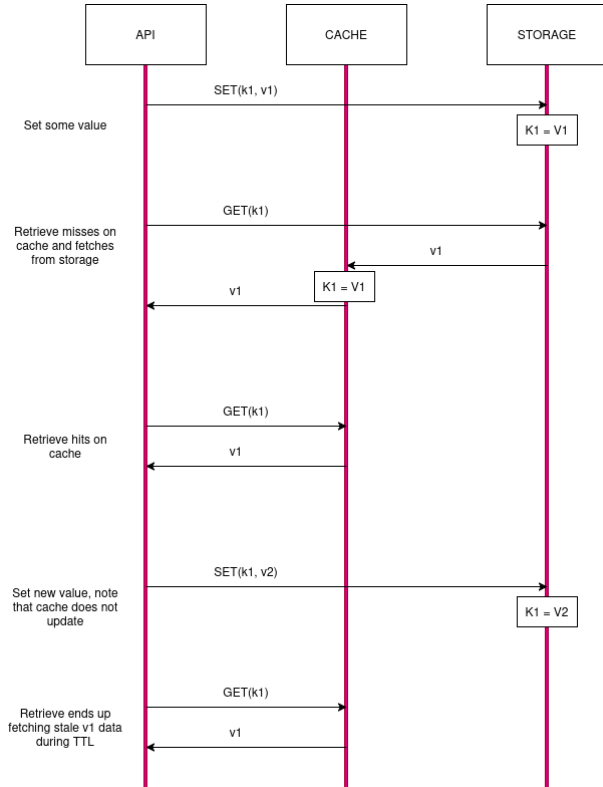


Fig. 2. TTL Caching

Here, *GET* retrieves the value for the given key, *SET* writes or updates the value for a given key, and *EVICT* removes an entry from the cache without affecting the storage.

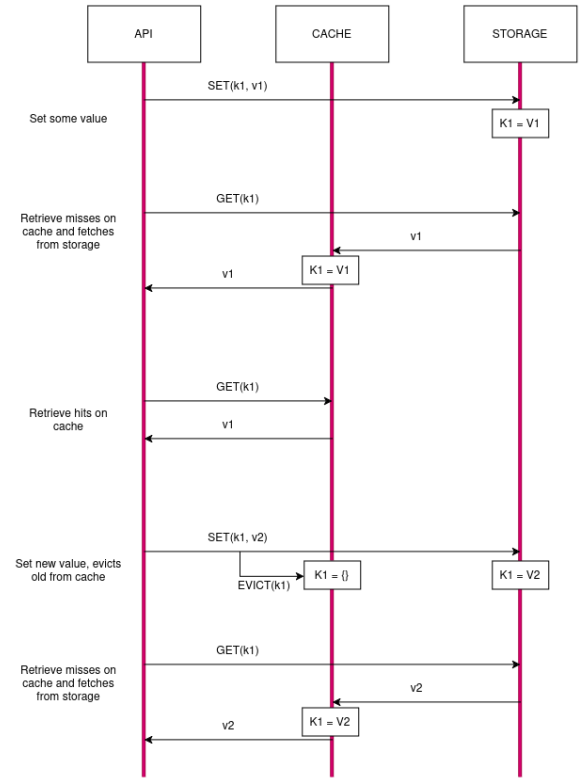


Fig. 3. Push-Evict Caching

IV. EARLY ENVIRONMENT AND SCOPE

A. Experimental Setup

We will implement and test the following configurations on personal hardware:

- 1) A "no-cache" baseline version.
- 2) A "TTL-cache" version.
- 3) A "PushEvict-cache" version.

Other tasks running on the same machine may interrupt or interfere with measurements; averaging a large number of samples per configuration is intended to yield numbers that are at least relevant for comparative purposes.

B. Metrics

We evaluate each configuration along two metrics:

Hit ratio and *miss ratio* measure how often a request is served from the cache versus requiring a fetch from the backend storage. This will show us the benefit of whether it is beneficial or not.

Freshness measures how outdated the data served by the cache is relative to the most recent write to the underlying data source. To specify: we are looking at freshness for the cache as a whole, not for the individual data elements, so it is expressed as a ratio rather than a per element measure. It is calculated as the number of fresh hits divided by the total

number of hits, where a fresh hit is a cache hit that matches the current value in the storage. This tells us what percentage of hits actually correspond to the correct, current data in storage.

C. Experiment Parameters

We plan to run X samples for each configuration, with data access patterns randomized to reflect write-heavy expectations [2].

V. EARLY TIMELINE

At the start we feel it's going to be important for us to understand and plan out on how we are going to measure our data within our metrics so the results from the data can be comprehensive for the experiment we want to conduct since we are on the same road as [3]

VI. DISCUSSION

We expect a clear separation in hit/miss ratio as write frequency increases: the TTL cache should keep a stable hit ratio, since the data is not evicted on write, while the PushEvict cache should see its hit ratio drop under write heavy conditions, as frequent evictions force the incoming requests back to storage as shown in Figure 3

Under write heavy conditions, this pattern repeats for every write, meaning the PushEvict cache rarely gets to serve a hit for a key that is written often.

We expect the TTL cache should show increasing staleness as write frequency rises connected to its TTL timing, while the PushEvict cache should maintain near constant freshness regardless of write frequency, since eviction is triggered by write itself rather than the TTL timing.

REFERENCES

- [1] J. Wang, "A survey of web caching schemes for the internet," *ACM SIGCOMM Computer Communication Review*, vol. 29, no. 5, pp. 36–46, Oct. 1999.
- [2] J. Yang, Y. Yue, and K. V. Rashmi, "A large-scale analysis of hundreds of in-memory key-value cache clusters at twitter," *ACM Transactions on Storage*, vol. 17, no. 3, pp. 1–35, Aug. 2021. DOI: 10.1145/3468521
- [3] B. Tripathi and M. Umar, "Experimental analysis of server-side caching for web performance," 2025.